

Incompatibility by Construction

What Cannot Be Fixed With More LLM

Registry: [\[HOWL-LLM-4-2026\]](#)

DOI: 10.5281/zenodo.20097126

Series Path: [\[HOWL-LLM-2-2026\]](#) → [\[HOWL-LLM-3-2026\]](#) → [\[HOWL-LLM-4-2026\]](#)

Date: May 2026

Domain: LLM Usage Methodology

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic’s Claude Opus 4.6.

Abstract

LLM-assisted software development operates under two success criteria that are incompatible by construction. The model’s success criterion is locally coherent output — code that is syntactically valid, responsive to the current prompt, and consistent with whatever context is loaded this turn. The engineer’s success criterion is total system correctness — a finished product where every component aligns with every other component across all conditions, including conditions no one enumerated. The first is a per-turn local property that the transformer architecture can provide. The second is a continuous global property that requires persistent constraint maintenance the architecture does not have. No sequence of locally successful outputs guarantees the global property, because the global property depends on cross-output consistency that no individual output can ensure. This paper identifies eighteen specific cases where the two criteria diverge, demonstrates why tests, specifications, and agentic workflows each fail to bridge the gap, and establishes that the incompatibility follows from what per-turn generation is — not from how good it currently is. The incompatibility cannot be fixed with more LLM. It can only be managed by an engineer who knows it exists and scopes their use of the tool accordingly.

1. What You Are Reading This For

You use LLMs for engineering work. You have had them produce modules that compile, pass tests, and meet the spec. You have also had your system break in ways that no individual module predicted — a resource leak that spans three modules, a concurrency failure that emerges only under composition, a security exposure that lives in the gap between what the spec said and what the system needed. You may have attributed these failures to insufficient testing, incomplete specifications, or the model not being capable enough yet.

This paper presents a different explanation. The failures are not gaps in capability. They are consequences of a structural mismatch between what the model optimizes for and what finished software requires. The model optimizes per-turn for local output quality. You optimize continuously for total system correctness. These are different optimization targets, and no amount of capability improvement, tooling, or workflow design changes one into the other, because the difference is between what per-turn generation provides and what a finished system demands.

When you finish this paper, you should be able to identify both success criteria in any LLM-assisted workflow, predict where they diverge before the divergence produces a failure, and scope your use of the tool to tasks where the divergence is manageable. You should also be able to explain to others why the claim that more capable models will close this gap requires a mechanism that does not exist in the current architecture and has not been demonstrated in any proposed architecture.

2. The Equation

When an LLM generates code, the output is a function of three inputs: the model's training weights, the context currently loaded in the session, and the goal supplied in the prompt. Training + context + goal = output. This equation executes fresh on every turn. There is no carryover mechanism that preserves decisions from one turn to the next except insofar as previous output remains in the context window, where it competes with everything else in context for influence on the current generation.

Five properties of the transformer architecture shape what this equation can and cannot produce.

The model has no persistent state. Every turn generates from the current context. There is no mechanism that says "I decided X on the previous turn, so I must remain consistent with X." Consistency with earlier output is maintained only when that output is still in context and the pattern matching happens to reproduce compatible decisions.

The model has no constraint checker. If constraints are stated in the prompt, they enter context and probabilistically influence generation. There is no verification step that rejects output violating a stated constraint. The model can violate a constraint from 800 tokens ago because the local probability distribution at the current generation step does not weight that constraint strongly enough.

Attention degrades across context length. The model is effective at local coherence — syntax, function-level structure, nearby relationships — and degrades on global coherence — system-wide invariants, cross-cutting concerns, constraints that must hold across the entire output. Information at the beginning and end of context receives more attention than information in the middle.

Generation trends toward the training-weight median. Every output drifts toward the statistical center of the training corpus — predominantly small projects, tutorials, example code, Stack Overflow answers — unless context actively steers it elsewhere. When

context’s influence attenuates, training priors reassert.

The model has no knowledge of the engineer’s system. It knows nothing about the specific codebase, the constraint surface, the architectural decisions, or the conventions, except what the engineer loads into context this turn.

The goal term — the engineer’s intent, what they want the output to accomplish — is the engineer’s irreducible contribution. Without it, the model produces the training-weight median: what most code looked like across the training corpus. The goal is what differentiates “write module X following convention Y respecting constraint Z” from “produce the most probable code given this context.” The engineer supplies the goal. The model cannot generate it, because generating a goal requires knowing what the system needs, and the model has no persistent model of the system.

The goal rebuilds every turn. The engineer cannot supply a goal hierarchy — present task (write module X), project trajectory (finish system Y), principles and constraints (avoid A, ensure B, align with C) — and have all levels govern generation simultaneously. The context window is finite, attention degrades, and the more the engineer loads into the goal term, the more each piece competes for influence on the output distribution. The model can serve the bottom level of the hierarchy on a given turn if scoped tightly enough. The upper levels remain with the engineer and cannot be delegated, because the architecture does not provide for holding them across turns or subordinating local generation to goals that are not actively present in the prompt.

3. The Model’s Success Criterion

The model succeeds when it produces output that is locally coherent within the generation pass, syntactically valid, consistent with this turn’s context, and responsive to this turn’s goal.

This is not a low bar. Within a single generation pass, the model can produce genuinely good code — correct logic, appropriate patterns, clean structure, accurate use of APIs and language features. The output can be better than what many engineers would produce for the same isolated task, because the model draws on patterns across millions of codebases and selects the statistically strongest match for the given context.

The model maintains this quality up to approximately 1,200 lines of real code in a single generation pass. This is an empirical observation from fifteen months of daily use across multiple model generations. Beyond that length, attention degradation under constraint load causes the model to begin violating its own earlier decisions — using different conventions at line 1,100 than it established at line 100, importing across boundaries it was told to respect, adopting patterns from training data that conflict with patterns it was following from context. The ceiling has not moved across model generations. Better models improve quality within the ceiling. They do not extend it.

The micro-decisions within each generation — which variable name, which control flow pattern, which error handling strategy, which structural idiom — are made by

sampling from the model's probability distribution. Each micro-decision reflects the training-weight median steered by context. The decisions are locally plausible. They may even be locally optimal by the standards of the training corpus. They are not the engineer's decisions, and they are not made with reference to anything outside the current generation's context.

The model's success criterion, stated precisely: produce locally plausible output that is coherent within this generation pass and responsive to this turn's inputs. Everything the architecture provides — pattern matching, statistical selection, context-steered generation — serves this criterion. Nothing the architecture provides extends beyond it.

4. The Engineer's Success Criterion

A finished software system must be 100% aligned with itself. Not mostly aligned. Not aligned per-module. Totally aligned — every component consistent with every other component across all conditions, including conditions no one enumerated in advance.

This is not an aspiration. It is the success criterion for software that does not have bugs, does not crash, does not leak memory, does not expose security vulnerabilities, does not corrupt state, does not deadlock under load, does not degrade unpredictably under partial failure. Software that falls short of total alignment fails in the gaps — the places where one component's assumptions do not match another component's behavior, where a resource lifetime crosses a boundary the designer did not account for, where an error path leads to a state that no module was designed to recover from.

The complete case is the success criterion. Not writing a module. Not writing a set of modules. Not passing tests. Not meeting a spec. The whole system holding together under all conditions.

What total alignment requires:

Convention consistency across every file in the codebase, enforced not by a single generation pass but by every decision made over the life of the project. Naming, patterns, idioms, architectural boundaries — all must hold everywhere, including in files written months apart.

Memory safety across every lifetime across every path. Not within a function — across every function that touches a resource, across every module that shares ownership, across every execution path that might free, retain, or transfer a resource. Leaks and use-after-free happen at boundaries, not within functions.

Error handling as a system-wide contract. Every error handled exactly once, at the right level, with the right recovery or propagation, no error path leaving the system in an inconsistent state. The contract spans every module and every call path.

Concurrency correctness as a whole-system property. Not per-module synchronization — system-wide lock ordering, freedom from deadlock under composition, correct behavior when independently synchronized modules interact.

Security beyond what any specification enumerates. Input validation, bounds checking, timing side channels, information leakage through error messages, privilege escalation through unexpected state combinations. The spec covers intended behavior. Security requires covering unintended behavior — what the system must not do, which is unbounded and cannot be fully enumerated.

Performance under composition, not in isolation. A module that is performant alone can become a bottleneck when composed with the rest of the system. Cache behavior, contention patterns, allocation pressure, pipeline stalls — these emerge from how modules interact under real workloads.

Graceful degradation designed across the whole system. How the system behaves when a dependency is unavailable, when memory is low, when a network call times out, when a disk is full. These behaviors must be designed, not discovered.

Backward compatibility across the project's timeline. The current interface must be compatible with past callers and must not foreclose future evolution. Correctness at a point in time is insufficient. Correctness across the project's history and future is required.

Build and dependency coherence. Every dependency compatible with every other dependency, every version constraint satisfiable, every build configuration consistent across the system.

The engineer carries these requirements as a continuous constraint model in their head. The model grows with every decision, includes negative constraints (what to avoid, what to leave out, what to constrain preemptively), and governs every line of code whether written by the engineer or generated by a tool. The model is not a checklist. It is a living system of interrelated constraints that the engineer maintains across the entire life of the project.

5. The Incompatibility

The model optimizes per-turn for local output quality. The engineer optimizes continuously for total system correctness. These are not points on a spectrum. They are structurally different optimization targets.

Local output quality is a per-turn property. The architecture provides it through pattern matching, statistical selection, and context-steered generation. It operates within a single generation pass, on whatever context is loaded, producing output that is coherent within that pass. It is real and genuinely useful. It is also the ceiling of what the architecture can optimize for.

Total system correctness is a continuous global property. It requires persistent constraint maintenance — a model of the entire system's invariants, checked against every decision, maintained across every change, growing with the system over its lifetime. The transformer architecture does not provide persistent state, does not provide constraint checking, and does not maintain anything across turns. There is no mechanism in the archi-

ture that produces total system correctness, because the architecture operates per-turn and total system correctness is not a per-turn property.

The incompatibility is by construction. It follows from what per-turn generation is, not from how good the model currently is. A more capable model produces better local output — fewer syntax errors, more accurate pattern matching, better convention following within each generation. It does not produce global correctness, because global correctness requires mechanisms the architecture does not have regardless of capability level.

No sequence of locally successful outputs guarantees the global property. Each output can be individually excellent — correct logic, clean structure, good conventions, passing tests. The system composed of those outputs can still fail, because the global property depends on cross-output consistency: every output's decisions must be compatible with every other output's decisions, across all conditions, including conditions that were not present in any individual output's context. No individual output can ensure this compatibility, because it was generated without knowledge of the full set of outputs it must be compatible with.

This is the structural claim of the paper. The model's reachable success criterion and the engineer's required success criterion are different in kind, not in degree. The gap between them is not a capability gap that closes with scale. It is an architectural gap between what per-turn local generation provides and what continuous global correctness requires.

6. The Eighteen Cases

The incompatibility manifests in eighteen specific cases, grouped into three categories: boundary cases where local correctness does not ensure system correctness, temporal cases where per-turn generation cannot address properties that span the project's timeline, and structural cases where the model's optimization target actively opposes the engineer's.

Boundary Cases

Memory safety across modules. The model produces a function with correct allocation and deallocation. It cannot verify that the allocation pattern is compatible with patterns in every other module sharing resources. Leaks happen at boundaries — one module allocates, another takes ownership, a third frees under a condition the first did not anticipate. The model sees one function. The engineer must see every lifetime across every path.

Error handling across call paths. The model generates error handling per function — check this return value, propagate this error code. Error handling in a finished system is a contract: every error handled exactly once, at the right level, with correct recovery or propagation, no error path leaving the system inconsistent. The model handles errors locally. The engineer must ensure the contract holds across every module and every call path.

Concurrency across lock orderings. A module can be correctly synchronized internally and deadlock when composed with another correctly synchronized module because their

lock orderings are incompatible. The model cannot reason about lock ordering across modules generated in different sessions with different contexts.

Performance under composition. A module performant in isolation can bottleneck the system when composed with other modules. Cache behavior, contention patterns, allocation pressure, pipeline stalls emerge from interaction under real workloads. The model optimizes each module locally. The engineer must optimize the system globally.

Graceful degradation across the system. The model produces code that works under normal conditions. How the system behaves under partial failure — unavailable dependencies, low memory, network timeouts, full disks — must be designed across the whole system, not per module. Each module's degradation behavior must be compatible with every other module's, producing a system that fails gracefully rather than unpredictably.

Temporal Cases

Convention consistency across the codebase's lifetime. The model follows conventions in context for this generation. Across a codebase built over months or years, conventions must hold everywhere — including files written months apart, in sessions with different contexts, reflecting different states of the model's probability distribution. The model matches what it sees this turn. The engineer enforces what must be true across every file the project has ever contained.

API evolution and backward compatibility. The model generates against the current interface. The engineer considers what the interface was, what it will become, what callers depend on, what changes break downstream consumers. The model's success is current correctness. The engineer's success is correctness across the project's timeline.

Cross-generation coherence. Each generation pass is a separate ballistic landing — internally coherent, shaped by its own session state and context. Multiple landings across a codebase produce modules shaped by different probability distribution states, different context loads, different positions in the model's training distribution. The model succeeds per landing. The engineer succeeds only when all landings cohere as a system. No mechanism in the generation process ensures cross-landing coherence.

Build and dependency coherence. The model produces code that compiles this turn. Whether dependencies it introduced are compatible with existing dependencies, whether version constraints conflict, whether the build system handles the new module's requirements — these are properties that span the project's history of dependency decisions and cannot be evaluated within a single generation.

Structural Cases

Tests passing versus the system being correct. The model can generate code that satisfies enumerated test cases. The complete case includes everything not tested — race conditions under specific load patterns, resource exhaustion, error paths that combine across module boundaries, state corruption through sequences no individual test exercises. The model's reachable criterion is test passage. The engineer's required criterion is total correctness including unenumerated cases.

Spec compliance versus security. A module can implement an API spec perfectly and expose attack surface the spec never addressed. Input validation, bounds checking, timing side channels, information leakage through error messages, privilege escalation through unexpected state combinations. The spec covers intended behavior. Security requires covering unintended behavior, which is unbounded.

Training-weight median versus system-specific correctness. Without active context steering, the model produces the most probable output from its training distribution — what most code looked like. The model’s implicit target is proximity to the training median. The engineer’s target is alignment with their specific system, which may diverge arbitrarily far from the training median. Every point of divergence is a point where the model’s success and the engineer’s success are in direct opposition.

Expansion versus precision. The model’s training rewarded longer, more thorough responses. Its implicit success criterion includes expansion — more explanation, more auxiliary code, more coverage. The engineer’s success criterion is precision — exactly what was needed, nothing more. The model succeeds by its training metric when it produces 400 lines for a 50-line request. The engineer succeeds only when they receive exactly what they asked for.

Ballistic micro-decisions versus engineered micro-decisions. Every variable name, control flow branch, and error handling choice in generated code was made by a probability distribution. Each reflects the training-weight median steered by context, not the engineer’s system-specific reasoning. The model’s criterion is that each micro-decision be locally plausible. The engineer’s criterion is that each micro-decision be globally correct for this specific system. Plausible and correct are different properties, and the gap compounds across hundreds of micro-decisions per generation.

Comprehension as a success criterion. The model succeeds by producing working code. The engineer succeeds only when they understand the code well enough to maintain, modify, and debug it under pressure. Code the engineer possesses but does not understand is not finished — it is a liability. The model has no comprehension requirement in its success criterion. The engineer’s criterion is incomplete without it.

The goal hierarchy. The model can serve one goal per turn. The engineer operates under a goal hierarchy: present task, project trajectory, principles and constraints. The model cannot hold this hierarchy because it rebuilds every turn. The engineer’s success requires all levels satisfied simultaneously. The model can address the bottom level if scoped tightly. The upper levels cannot be delegated because the architecture does not provide for holding them.

Per-turn rebuild versus persistent constraint maintenance. The equation rebuilds every turn. The engineer’s constraint model is continuous and cumulative — every decision made across the project’s life remains active. The model’s constraint model is whatever fits in context this turn. The model succeeds by producing output consistent with this turn’s context. The engineer succeeds only when every decision ever made in the project remains consistent with every other decision.

Each case follows from the same root: the model operates per-turn on local context opti-

mizing for local output quality, the engineer operates continuously on the whole system optimizing for total system correctness, and the two optimization targets are incompatible by construction.

7. Why Tests Don't Close the Gap

Tests are the primary mechanism the industry relies on to ensure correctness of AI-generated code. The argument: if the generated code passes comprehensive tests, the code is correct. The argument fails because tests address the model's success criterion without reaching the engineer's.

Tests enumerate known conditions and verify behavior under those conditions. The complete case includes unenumerated conditions — the conditions that no one thought to test because they emerge from interactions the test designer did not anticipate. Race conditions under specific load patterns that no unit test exercises. Resource exhaustion when three modules that are individually efficient compete for the same pool under contention. Error paths that combine across module boundaries in sequences that no individual module's tests cover. State corruption through timing that only occurs when independently tested modules interact under production workloads.

Tests verify behavior. They do not verify constraint preservation, convention consistency, or architectural alignment. A module can pass all its tests while using a different naming convention than every other module in the system. A function can pass all its tests while importing across an architectural boundary the system relies on for isolation. A component can pass all its tests while using an allocation pattern that is incompatible with the allocator strategy the rest of the system depends on.

The model can generate code that passes all tests while violating properties that tests do not cover. This is not a failure of the tests or the model. It is a category mismatch. Tests verify local behavior — does this function produce the right output for the given input. The engineer's success criterion is global correctness — does this system hold together under all conditions. These are different properties. Testing the first does not establish the second. The class of properties that tests do not cover — cross-module invariants, system-wide resource management, emergent behavior under composition — is exactly the class that causes system-level failures.

Adding more tests narrows the gap incrementally without closing it. The unenumerated space is larger than the enumerated space, because the enumerated space is what the designer thought of and the unenumerated space is everything else. Better test coverage is valuable. It does not convert the model's success criterion into the engineer's.

8. Why Specs Don't Close the Gap

Specifications are the industry's answer to "how do we tell the model what to build." The argument: if the spec is comprehensive enough, the model generates code that meets

the spec, and spec-compliant code is correct. The argument fails for the same structural reason as the testing argument.

A spec is an enumeration of intended behavior. It states what the system should do — accept this input, produce this output, handle this error, expose this interface. A good spec is precise, complete within its scope, and testable. A good spec is also finite. It enumerates intended behavior. It cannot enumerate all unintended behavior the system must avoid, because unintended behavior is unbounded.

Security is the clearest example. A spec describes the API: these endpoints, these parameters, these responses, these authentication requirements. The spec does not enumerate every input validation vulnerability, every timing side channel, every information leakage path through error messages, every privilege escalation through unexpected state combinations. These are properties the system must have — not properties the system must do. The spec covers doing. Security requires not-doing across an unbounded space of things the system must not do.

Performance under composition is another example. The spec describes what each module does. It does not describe how each module’s resource usage interacts with every other module’s resource usage under every workload pattern. A module can meet its spec while introducing cache contention that halves the system’s throughput. The spec didn’t cover cache behavior because the spec describes function, not interaction.

Memory lifecycle is another. The spec describes the interface: allocate here, free here, these are the ownership semantics. It does not describe every path through the system where ownership transfers implicitly, where a resource outlives the scope the spec assigned it to, where a module holds a reference the spec’s lifetime model did not account for.

The gap between spec and complete case is where engineering judgment lives. The engineer reads the spec and also considers everything the spec does not say — the security implications, the performance interactions, the resource lifecycle concerns, the degradation behaviors, the evolutionary constraints. This judgment requires a model of the entire system that the spec does not contain and the LLM does not have. The model can meet the spec. The engineer must meet the spec plus everything the spec didn’t say.

9. Why Agents Make It Worse

Agentic workflows — tools where the model plans, executes, tests, and iterates with minimal human intervention — are positioned as the solution to the limitations of simple chat-based generation. The argument: the agent iterates until the tests pass and the spec is met, so the result is correct. The argument inherits the failures of Sections 7 and 8 and adds a new one.

Each autonomous step in an agentic workflow is a turn where the agent supplies its own subgoal. The engineer’s goal entered once, at the top, as the initial prompt. Every subsequent step is a subgoal generated by the model from its training weights — the most

probable decomposition, the most probable file to edit, the most probable pattern to apply. Each subgoal is the training-weight median’s version of what should happen next, not the engineer’s.

The engineer’s goal enters the equation as training + context + goal = output. When the agent generates its own subgoals, the equation becomes training + context + training-derived subgoal = output. The engineer’s goal has been replaced by a training-weight proxy. Each autonomous step moves the optimization target further from the engineer’s success criterion and closer to the model’s — locally plausible output that passes the tests the agent can run.

Each step that passes tests and meets spec reinforces the illusion of correctness. The agent reports success. The tests pass. The spec is satisfied. The system-level gaps — cross-module constraint violations, convention drift, architectural boundary crossings, resource lifecycle mismatches — accumulate undetected because the agent’s verification loop checks against tests and spec, which are the model’s success criterion, not the engineer’s.

The more autonomous the workflow, the more decisions are made without the engineer’s constraint model, and the more the complete case is violated in ways that will not surface until the system is under production load or until the engineer attempts a modification that touches the assumptions the agent’s decisions were built on. Autonomy does not close the incompatibility between the two success criteria. It removes the engineer from the point where they would have caught the divergence — the point where they look at the output, compare it against their knowledge of the whole system, and identify where the model’s locally plausible decision conflicts with a global constraint the model did not have.

The industry sells autonomy as the premium feature. For systems where the complete case matters, autonomy is the mechanism that maximizes the distance between the model’s success criterion and the engineer’s.

10. What Cannot Be Fixed With More LLM

The incompatibility is not a capability gap. It is a structural mismatch between what per-turn local generation optimizes for and what finished software requires.

More capable models improve output quality within each turn. Better syntax, better pattern matching, more accurate convention following, fewer trivial errors, more sophisticated handling of complex logic within a single generation pass. These are improvements to the model’s success criterion — better local output quality. They are real and valuable. They make each individual generation more useful as raw material.

They do not change the per-turn structure. A more capable model still rebuilds from training + context + goal every turn. It still has no persistent state across turns. It still has no constraint checker that verifies output against a system-wide model. It still cannot hold the engineer’s goal hierarchy. It still generates micro-decisions from the training-weight distribution. It still trends toward the training median when context’s influence attenuates.

The improvements are within the architecture’s capability. The incompatibility is between the architecture’s capability and what finished software requires.

Larger context windows are the most common proposed remedy. The argument: if the context is large enough to hold the entire system, the model can maintain global constraints. The argument fails on two grounds. Attention degrades across context length — information in the middle of context receives less attention than information at the edges, and at system scale the amount of constraint-relevant information exceeds what attention can reliably weight. Loading an entire codebase into context does not mean the model holds the entire codebase’s constraints active during generation. It means the constraints enter the probability distribution at reduced weight, where they compete with training priors and lose at exactly the points where they need to win.

The second ground is that the constraint model is not the code. The code is an expression of the constraint model, but the constraint model includes negative constraints (what was deliberately not done), historical constraints (why a decision was made), evolutionary constraints (what must remain compatible with past and future states), and architectural constraints (what the code’s structure implies about what changes are safe). These are not in the code and cannot be loaded into context from the code. They exist in the engineer’s head as the accumulated residue of every decision made across the project’s life. No context window contains them because they were never written down — they are the engineering judgment that governs the project.

Better training helps within the same limits. A model trained on higher-quality code produces higher-quality local output. It does not produce global correctness, because global correctness is not a property of training data — it is a property of a specific system’s internal consistency, which no training data can supply.

Scaling addresses the model’s success criterion. The engineer’s success criterion is orthogonal to it.

11. The Engineer’s Irreducible Role

The complete case lives in the engineer’s head as a continuous constraint model. The model grows with every decision made across the project’s life. It includes positive constraints — what the system must do, how components must interact, what conventions must hold. It includes negative constraints — what the system must not do, what patterns to avoid, what dependencies to refuse. It includes temporal constraints — what the system was, what it must remain compatible with, what it will need to become. The model is not a document. It is not a spec. It is the accumulated understanding that the engineer maintains and updates continuously.

This constraint model governs every line of code in the project, whether written by the engineer or generated by a tool. When the engineer writes code by hand, the constraint model shapes every micro-decision — the variable name, the control flow, the error handling, the structural idiom. The decisions emerge from the constraint model automatically, as a byproduct of the engineer’s understanding. When a tool generates code, the constraint

model governs the evaluation of that code — every micro-decision checked against the engineer’s knowledge of what the system requires. The constraint model is the same in both cases. The difference is whether it governs generation or governs evaluation.

This role cannot be delegated to the model. Not because the model is insufficiently capable, but because the role requires exactly what the model does not have: persistent state across the project’s lifetime, a constraint model that grows with every decision, negative knowledge about what was deliberately excluded, and the goal hierarchy that subordinates every local decision to the project’s trajectory and principles. These are not features the model is missing. They are properties of the engineer’s continuous relationship with the system — a relationship the model cannot have because it rebuilds from scratch every turn.

The method documented in the companion papers — calibrate the session, extract bounded output, refine by hand — works because it respects this division. The model contributes raw material within its per-turn capability. The engineer contributes the global alignment that the model cannot. The model’s output is useful because the engineer’s constraint model evaluates it. Without the engineer’s constraint model, the output is locally plausible code with no guarantee of system-level correctness. The value is in the combination: the model’s speed and pattern coverage, filtered through the engineer’s constraint knowledge. Neither alone produces the result.

Workflows that blur this division — that delegate more decisions to the model, that reduce the engineer’s involvement in evaluating output, that automate the integration of generated code without constraint checking — accumulate the incompatibility as technical debt. The debt is invisible in the short term because each individual output passes its local checks. The debt surfaces when the system must change, when a failure traces to an assumption the model made that the engineer would not have made, when the accumulated micro-decisions from hundreds of separate generation passes produce emergent behavior that no individual pass anticipated.

The engineer’s role is irreducible because the complete case requires it. Not as oversight. Not as review. As the continuous constraint maintenance that is the difference between a collection of locally correct modules and a finished system.

12. What You Can Now Do

If the transmission worked, you can now identify the two success criteria operating in any LLM-assisted workflow. You can name the model’s criterion — locally coherent output, responsive to this turn’s inputs, plausible by training-weight standards — and the engineer’s criterion — total system correctness across all conditions including unenumerated ones. You can explain why these are different in kind, not in degree.

You can predict where the two criteria diverge before the divergence produces a failure. When a generation involves memory safety across module boundaries, error handling as a system-wide contract, concurrency under composition, performance under real workloads, security beyond the spec, convention consistency across the codebase’s lifetime —

you know these are cases where the model's success does not imply the engineer's success. You can scope the generation to avoid the cases where divergence is most costly, or allocate time for the constraint verification that catches divergence after generation.

You can refuse to delegate tasks where only the engineer's success criterion applies. Architectural decisions, cross-module constraint design, system-level correctness verification, negative constraints, evolutionary planning — these require the persistent constraint model that the engineer carries and the model does not have. Delegating them produces output that is locally plausible and globally unverified.

You can explain why tests and specs don't close the gap. Tests verify enumerated conditions. Specs describe intended behavior. The complete case includes unenumerated conditions and unintended behavior the system must avoid. The gap between what tests and specs cover and what the complete case requires is where the two success criteria diverge most dangerously, because the divergence is invisible to the verification tools the industry relies on.

You can explain why agentic workflows increase the incompatibility rather than resolving it. Each autonomous step replaces the engineer's goal with a training-derived subgoal, moving the optimization target from the engineer's criterion to the model's. More autonomy means more decisions made without the engineer's constraint model, which means more system-level gaps accumulating behind the appearance of passing tests.

You can strip claims that more capable models will close the gap by asking for the mechanism. What mechanism converts per-turn local generation into continuous global constraint maintenance? What mechanism gives the model persistent state across the project's lifetime? What mechanism installs the engineer's goal hierarchy into an architecture that rebuilds every turn? If the answer is "more capability," you can identify this as an improvement to the model's success criterion — better local output — that does not address the engineer's success criterion, which is orthogonal to it.

You can use this understanding to work productively with LLMs. The model is genuinely useful within its success criterion. The engineer who knows the boundary between the two criteria scopes their use of the tool to the model's side, handles everything on the engineer's side themselves, and extracts value from every session without accumulating the debt that comes from confusing one criterion for the other.

13. Closing

The model is good at what it optimizes for. The engineer needs something different from what the model optimizes for. The incompatibility is by construction — it follows from what per-turn generation is, not from how good it currently is.

The industry optimizes for the model's success criterion. More output, faster, passing more tests, meeting more specs, with less human involvement. Every metric the industry reports measures the model's criterion: time-to-output, test pass rate, spec compliance,

lines generated. These metrics improve with every model generation. They will continue to improve. They measure the wrong thing.

The engineer’s success criterion — a finished system that holds together under all conditions — is orthogonal to these metrics. A system can score perfectly on every industry metric and fail in production because the metrics measure local output quality and the failure is a global property. The metrics will not capture the failure because the failure lives in the gap between the two success criteria, and the metrics only measure one of them.

Knowing the incompatibility does not make the model less useful. It makes the model more useful, because the engineer who knows the boundary does not waste time or accumulate debt on the wrong side of it. They use the model for what per-turn generation is good at — pattern assembly, design-space search, boilerplate, first-pass implementations, enumeration, comprehension mirroring. They handle what per-turn generation cannot provide — global constraint maintenance, cross-module coherence, architectural judgment, the complete case. The division is clean, productive, and stable across model generations, because it follows from the architecture rather than from the current capability level.

The complete case is the engineer’s. It was always the engineer’s. The model did not change that. Knowing that the model did not change it is what makes the model safe to use.

[[HOWL-LLM-4-2026](#)] Incompatibility by Construction. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM-4-2026>

[[HOWL-LLM-2-2026](#)] Calibrate, Extract, Refine. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM-2-2026>

[[HOWL-LLM-3-2026](#)] Riding the Rocket. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-3-2026>